Laura Hietala

Oppimistehtävä 6

Tietojenkäsittelyn koulutus
TKMI23KV

17.12.2023

Sovellusohjelmointi 1
Janne Turunen

Tehtäväraportti

Tehtävänä on toteuttaa React-, MUI- ja React Router -tekniikoin sovellus, jolla käyttäjä voi pitää kirjaa erilaisiin liikunnallisiin aktiviteetteihin käyttämästään ajasta. Liikunnan sisältö voi olla mitä tahansa (esim. kävely, juoksu, hiihto, sulkapallo jne.). Tietoja ei tarvitse tallentaa palvelimelle, localStorageen tms. vaan päiväkirja voi tyhjentyä uudelleen käynnistämisen yhteydessä.

Sovelluksen käyttöliittymäratkaisut ja navigaation voi toteuttaa vapaasti, soveltaen kuitenkin Material-UI-komponentteja (vaatimukset suluissa).

GitHub*: https://github.com/xamk-so1/oppimisteht-v-6-elahi001xamk*

Perusvaatimukset sovellukselle

Vaatimus: Sovelluksella on mahdollista kirjata uusi aktiviteetti omassa näkymässään. Käyttäjä kirjaa aktiviteetista alkamis- ja päättymisajankohdan minuutin tarkkuudella (DateTimePicker) sekä kuvaustekstin (TextField) aktiviteetin sisällöstä (kävely, juoksu jne.).

Ratkaisu: UusiAktiviteetti- komponentissa on toteutettu uuden aktiviteetin kirjaaminen. Käyttäjä voi syöttää alkamis- ja päättymisajankohdan minuutin tarkkuudella (DateTimePicker-komponentti riveillä 63-68 ja 70-75), kuvaustekstin (TextField-komponentti riveillä 77-82) ja tallentaa aktiviteetin (Button-komponentti riveillä 84-86). Apuna on tiedoston alussa määritellyt tilamuuttujat alkuaika, loppuaika ja kuvaus, jotka tallentavat aktiviteetin alkamisajan, päättymisajan ja kuvaustekstin.

```
<LocalizationProvider
dateAdapter={AdapterDateFns}
adapterLocale={fi}
>

 <Typography variant="h3" style={{ margin: "20px", fontFamily: "Gabriola"}}>Uusi Aktiviteetti</Typography>

  <DateTimePicker sx={{ margin: "20px", backgroundColor: "#d2e8d1"}}
   label="Aktiviteetin alkuaika"
   value={alkuaika}
   onChange={(newValue) => setAlkuaika(newValue || new Date())}
   slots={(params : any) => <TextField {...params} />}
   />

  <DateTimePicker sx={{ margin: "20px", backgroundColor: "#d2e8d1"}}
   label="Aktiviteetin päättymisaika"
   value={loppuaika}
   onChange={(newValue) => setLoppuaika(newValue || new Date())}
   slots={(params:any) => <TextField {...params} />}
  />

  <TextField
   style={{margin: "20px", backgroundColor: "#d2e8d1"}}
   label="Kuvaus"
   value={kuvaus}
   onChange={(e) => setKuvaus(e.target.value)}
  />

  <Button sx={{ margin: "20px", backgroundColor: "#8f6fbd"}}  variant="contained" onClick={tallennaAktiviteetti}>
   Tallenna aktiviteetti
  </Button>

  {virhe && <Typography color="error">{virhe}</Typography>}

 </LocalizationProvider>
```

Riviltä 24 alkava tallennaAktiviteetti-funktio suoritetaan, kun käyttäjä tallentaa uuden aktiviteetin ja sen avulla uusi aktiviteetti lisätään.

```
const tallennaAktiviteetti = () => {
  if (!alkuaika || alkuaika > new Date()) {
    setVirhe('Aktiviteetti ei voi alkaa tulevaisuudessa.');
    return;
  }

  if (!loppuaika || loppuaika > new Date()) {
    setVirhe('Aktiviteetti ei voi päättyä tulevaisuudessa.');
    return;
  }

  if (kuvaus.trim() === '') {
    setVirhe('Kuvaus ei voi olla tyhjä.');
    return;
  }

  const uusiAktiviteetti: Aktiviteetti = {
      alkuaika: alkuaika || new Date(),
      loppuaika: loppuaika || new Date(),
      kuvaus,
    };

 lisaaAktiviteetti(uusiAktiviteetti);
  setAlkuaika(new Date());
  setLoppuaika(new Date());
  setKuvaus('');
  setVirhe('');
 };
```

App-komponentissa on reititys (Routes + Routet riveillä 29-38), jonka ansiosta uusi aktiviteetti voidaan kirjata omassa näkymässään (reititys riveillä 34-37). UusiAktiviteetti-komponentissa kirjatut aktiviteetit tallennetaan App-komponentissa sijaitsevaan aktiviteetit-tilaan (rivi 32).

```
29        <Routes>
30          <Route
31            path="/"
32            element={<Yhteenveto aktiviteetit={aktiviteetit} />}
33          />
34          <Route
35            path="/uusi"
36            element={<UusiAktiviteetti lisaaAktiviteetti={lisaaAktiviteetti} />}
37          />
38        </Routes>
```

Vaatimus: Kaikki aktiviteettiin liittyvät tiedot ovat pakollisia ja tietojen puuttumisesta ilmoitetaan asianmukaisin virheilmoituksin. Aktiviteetit kirjataan aina suorituksen jälkeen, joten varmista myös, ettei käyttäjä voi määritellä alkamis- tai päättymisajankohtaa tulevaisuuteen.

Ratkaisu: Pakollisuus tarkistetaan sivulla 2 näkyvässä koodissa riveillä 35-38. Apuna käytetään virhe-tilamuuttujaa, jonka tilaa muutetaan, jos kuvaus on tyhjä. Se antaa tällöin virheilmoituksen ”Kuvaus ei voi olla tyhjä.”. Riveillä 25-33 tarkistetaan, että aktiviteetin alkuaika ja loppuaika ei ole tulevaisuudessa ja jos ovat, tulee virheilmoitus ”Aktiviteetti ei voi alkaa/päättyä tulevaisuudessa.”. Jos joku näistä ehdoista toteutuu, ohjelman suoritus palaa välittömästi return-lauseen avulla, jolloin uusi aktiviteetti ei tallennu.

Vaatimus: Kaikkia kirjattuja aktiviteetteja voi tarkastella omassa listanäkymässään (List). Listaus voi olla joko aloitusnäkymässä tai omana näkymänään.

Ratkaisu: Tämä tapahtuu AktiviteettiLista-komponentissa. AktiviteettiListassa määritellään lista (rivit 22-31), joka sisältää ListItem-komponentin (rivit 24-29) jokaiselle aktiviteetille. ListItemit sisältävät ListItemText-komponentin (rivit 25-28), joka näyttää aktiviteetin kuvauksen sekä alkamis- ja päättymisajan. Listaa

ylläpidetään mappauksen avulla (rivi 23 →)

```
7  interface Aktiviteetti {
8    alkuaika: Date;
9    loppuaika: Date;
10   kuvaus: string;
11 }
12
13 interface AktiviteettiListaProps {
14   aktiviteetit: Aktiviteetti[];
15 }
16
17 const AktiviteettiLista: React.FC<AktiviteettiListaProps> = ({ aktiviteetit }) => {
18   return (
19     <div>
20       <Box style={{display: "flex", flexDirection: "column", flexWrap: "wrap", justifyContent: "space-evenly", padding: "30px"}}>
21       <Typography variant="h3" style={{ margin: "20px", fontFamily: "Gabriola"}}>Aktiviteetit</Typography>
22       <List style={{ margin: "20px"}}>
23         {aktiviteetit.map((aktiviteetti, index) => (
24           <ListItem key={index}>
25             <ListItemText
26               primary={aktiviteetti.kuvaus}
27               secondary={`Alkuaika: ${format(aktiviteetti.alkuaika,'d.M.yyyy HH:mm')} - Päättymisaika: ${format(aktiviteetti.loppuaika, 'd.M.yyyy HH:mm')}`}
28             />
29           </ListItem>
30         ))}
31       </List>
```

Lista aktiviteeteista muodostuu App-tiedostossa. Siellä AktiviteettiLista-komponentti renderöidään, ja se saa propseina aktiviteetit-listan, joka sisältää kaikki kirjatut aktiviteetit (rivi 40).

```
16  const App : React.FC = () : React.ReactElement => {
17
18    const [aktiviteetit, setAktiviteetit] = useState<Aktiviteetti[]>([]);
19
20    const lisaaAktiviteetti = (uusiAktiviteetti : Aktiviteetti) => {
21      setAktiviteetit([...aktiviteetit, uusiAktiviteetti]);
22    };
23
24    return (
25      <>
26
27        <Sivu>
28
29          <Routes>
30            <Route
31              path="/"
32              element={<Yhteenveto aktiviteetit={aktiviteetit} />}
33            />
34            <Route
35              path="/uusi"
36              element={<UusiAktiviteetti lisaaAktiviteetti={lisaaAktiviteetti} />}
37            />
38          </Routes>
39
40          <AktiviteettiLista aktiviteetit={aktiviteetit} />
41
```

Vaatimus: Sovelluksen aloitusnäkymässä on yhteenveto aktiviteetteihin käytetystä yhteisajasta. Aika ilmoitetaan kokonaisina tunteina ja minuutteina, esim. "4 tuntia ja 34 minuuttia".

Ratkaisu: Yhteenveto-komponentti ottaa vastaan aktiviteetit-propsin (rivi 16) ja laskee tämän perusteella aktiviteettien yhteiskeston tunteina ja minuutteina ja näyttää sen käyttöliittymässä. Laskenta tapahtuu laskeYhteisaika-funktiossa (rivit 17-31). Yhteenveto-komponentti lisätään aloitusnäkymään App-komponentissa, joka myös välittää aktiviteetit-listan Yhteenveto-komponentille (s. 4 alempi kuva, rivi 32). Lopputulos on, että Yhteenveto-näkymässä näytetään aktiviteettien yhteiskesto tunteina ja minuutteina. Jos aktiviteetteja ei ole kirjattu, näytetään vain teksti "Ei vielä kirjattuja aktiviteetteja." Tämä tehdään ternary-operaattoria käyttäen riveillä 40-46.

```
16  const Yhteenveto: React.FC<YhteenvetoProps> = ({ aktiviteetit }) => {
17      const laskeYhteisaika = () => {
18          if (aktiviteetit.length === 0) {
19            return { tunnit: 0, minuutit: 0 };
20          }
21
22          const yhteisaikaMs = aktiviteetit.reduce(
23            (summa, aktiviteetti) => summa + (aktiviteetti.loppuaika.getTime() - aktiviteetti.alkuaika.getTime()),
24            0
25          );
26
27          const tunnit = Math.floor(yhteisaikaMs / (1000 * 60 * 60));
28          const minuutit = Math.floor((yhteisaikaMs % (1000 * 60 * 60)) / (1000 * 60));
29
30          return { tunnit, minuutit };
31        };
32
33        const { tunnit, minuutit } = laskeYhteisaika();
34
35
36    return (
37      <div>
38        <Box style={{display: "flex", flexDirection: "column", flexWrap: "wrap", justifyContent: "space-evenly", padding: "30px"}}>
39        <Typography variant="h3" style={{ margin: "20px",  fontFamily: "Gabriola"}}>Yhteenveto</Typography>
40        {aktiviteetit.length === 0 ? (
41          <Typography>Ei vielä kirjattuja aktiviteetteja.</Typography>
42        ) : (
43          <Typography style={{ margin: "20px"}}>
44            Aktiviteetteihin käytetty aika: {tunnit} tuntia ja {minuutit} minuuttia
45          </Typography>
46        )}
```

Lisävaatimukset sovellukselle

Tein tämän jälkeenpäin, siksi rivinumerot eivät täsmää aiempiin riveihin.

Vaatimus: Listanäkymässä aktiviteetit ovat järjestettynä lopetusajankohdan mukaan, viimeisin aktiviteetti ensin.

Ratkaisu: Luodaan rivillä 19 sortedAktiviteetit-muuttuja kopiona alkuperäisestä aktiviteetit-taulukosta ja järjestetään se sort-funktiolla loppumisajan mukaan niin, että viimeisin aktiviteetti on ensin. Järjestetyn taulukon perusteella rende-

röidään List-elementti alkaen riviltä 25, joka sisältää jokaisen aktiviteetin ListItem-elementtinä.

```
19  const sortedAktiviteetit = aktiviteetit.slice().sort((a, b) => b.loppuaika.getTime() - a.loppuaika.getTime());
20
21  return (
22    <div>
23      <Box style={{display: "flex", flexDirection: "column", flexWrap: "wrap", justifyContent: "space-evenly", padding: "30px"}}>
24      <Typography variant="h3" style={{ margin: "20px", fontFamily: "Gabriola"}}>Aktiviteetit</Typography>
25      <List style={{ margin: "20px"}}>
26        {sortedAktiviteetit.map((aktiviteetti, index) => (
27          <ListItem key={index}>
28            <ListItemText
29              primary={aktiviteetti.kuvaus}
30              secondary={`Alkuaika: ${format(aktiviteetti.alkuaika,'d.M.yyyy HH:mm')} - Päättymisaika: ${format(aktiviteetti.loppuaika, 'd.M.yyyy HH:mm')}`}
31            />
32          </ListItem>
33        ))}
34      </List>
```

Vaatimus: Aloitusnäkymän yhteenvedossa laskettu aktiviteettien yhteisaika ilmoitetaan erikseen viimeisen 7 vuorokauden osalta ja kaikkien tietojen osalta.

Ratkaisu: laskeYhteisaika -funktio (alkaa riviltä 17) saa kaksi aikapistettä (alkuAikaPiste ja loppuAikaPiste) ja laskee ajan niiden välillä. Funktio filteröi (rivit 22-26) ensin aktiviteetit, jotka sijoittuvat näiden aikapisteiden väliin. laskeKaikkiYhteen-funktio (rivit 40-43) laskee kaikkien aktiviteettien yhteisajan tähän päivään saakka. laskeViimSeitseman-funktio (rivit 45-49) taas laskee aktiviteettien yhteisajan viimeisen seitsemän vuorokauden ajalta.

```
17  const laskeYhteisaika = (alkuAikaPiste: number, loppuAikaPiste : number) => {
18      if (aktiviteetit.length === 0) {
19        return { tunnit: 0, minuutit: 0 };
20      }
21
22      const filterAktiviteetit = aktiviteetit.filter(
23        aktiviteetti => aktiviteetti.loppuaika.getTime() >= alkuAikaPiste
24        && aktiviteetti.alkuaika.getTime() <= loppuAikaPiste
25
26      );
27
28      const yhteisaikaMs = filterAktiviteetit.reduce(
29        (summa, aktiviteetti) => summa + (Math.min(aktiviteetti.loppuaika.getTime(), loppuAikaPiste) - Math.max(aktiviteetti.alkuaika.getTime(), alkuAikaPiste)),
30        0
31      );
32
33      const tunnit = Math.floor(yhteisaikaMs / (1000 * 60 * 60));
34      const minuutit = Math.floor((yhteisaikaMs % (1000 * 60 * 60)) / (1000 * 60));
35
36      return { tunnit, minuutit };
37    };
38
39
40    const laskeKaikkiYhteen = () => {
41      const nyt = new Date();
42      return laskeYhteisaika(0, nyt.getTime());
43    };
44
45    const laskeViimSeitseman = () => {
46      const nyt = new Date();
47      const viimeisetSeitseman = nyt.getTime() - (7 * 24 * 60 * 60 * 1000);
48      return laskeYhteisaika(viimeisetSeitseman, nyt.getTime());
49    };
50
51    const { tunnit: kaikkiTunnit, minuutit: kaikkiMinuutit } = laskeKaikkiYhteen();
52    const { tunnit: viimSeitTunnit, minuutit: viimSeitMinuutit } = laskeViimSeitseman();
53
```

Typography-komponentit (rivit 62-67) näyttävät kaikkien aktiviteettien ja viimeisen seitsemän vuorokauden aikana suoritettujen aktiviteettien yhteisajan:

